

Ejercicios de SQL

Práctica de funciones SQL Server

Tablas de referencia para los ejercicios:

Clientes(ID_Cliente, NombreCliente, LimiteCredito)
Productos(ID_Producto, NombreProducto, ID_Categoria, Precio, Stock)
Categorias(ID_Categoria, NombreCategoria)
Ventas(ID_Venta, ID_Cliente, Fecha)
DetalleVentas(ID_Venta, ID_Producto, Cantidad, PrecioUnitario)
Empleados(ID_Empleado, NombreEmpleado, Sueldo)
HorasTrabajadas(ID_Horas, ID_Empleado, Horas)

1. Función con CTE

Crear una función que devuelva los productos cuyo monto total vendido sea superior al promedio de monto vendido por producto. La función debe devolver:

ID_Producto
NombreProducto
MontoTotalVendido

El monto total vendido se calcula como:

$\text{SUM}(\text{DetalleVentas.Cantidad} * \text{DetalleVentas.PrecioUnitario})$

Restricciones:

- Debe usar CTE.
- No se puede usar TOP.
- No se puede usar MAX ni MIN.
- No se pueden modificar datos de las tablas base.
- Deben considerarse únicamente productos vendidos.

Resultado:

```
CREATE FUNCTION dbo.ProductosSobrePromedioVendido()  
RETURNS @Resultado TABLE (  
    ID_Producto INT,  
    NombreProducto VARCHAR(100),  
    MontoTotalVendido DECIMAL(12,2)  
)  
AS  
BEGIN  
    ;WITH TotalesPorProducto AS (  
        SELECT  
            P.ID_Producto,
```

```

        P.NombreProducto,
        SUM(DV.Cantidad * DV.PrecioUnitario) AS MontoTotalVendido
FROM Productos P
INNER JOIN DetalleVentas DV
    ON P.ID_Producto = DV.ID_Producto
GROUP BY
    P.ID_Producto,
    P.NombreProducto
),
PromedioGeneral AS (
    SELECT
        AVG(MontoTotalVendido) AS PromedioVendido
    FROM TotalesPorProducto
)
INSERT INTO @Resultado (
    ID_Producto,
    NombreProducto,
    MontoTotalVendido
)
SELECT
    T.ID_Producto,
    T.NombreProducto,
    T.MontoTotalVendido
FROM TotalesPorProducto T
CROSS JOIN PromedioGeneral P
WHERE T.MontoTotalVendido > P.PromedioVendido;

RETURN;
END;
Consulta de prueba:
SELECT *

FROM dbo.ProductosSobrePromedioVendido();

```

2. Función escalar

Crear una función escalar que reciba el ID de un cliente y devuelva el monto total facturado a ese cliente.

La función debe recibir: @ID_Cliente

Debe devolver: Monto total facturado

El total debe calcularse como:

$SUM(DetalleVentas.Cantidad * DetalleVentas.PrecioUnitario)$

Restricciones:

- Debe ser una función escalar.
- No puede devolver una tabla.
- Si el cliente no tiene ventas, debe devolver 0.

- No se puede usar cursor.
- No se pueden modificar datos.

Resultado:

```
CREATE FUNCTION dbo.TotalFacturadoCliente (
    @ID_Cliente INT
)
RETURNS DECIMAL(12,2)
AS
BEGIN
    DECLARE @Total DECIMAL(12,2);

    SELECT
        @Total = COALESCE(SUM(DV.Cantidad * DV.PrecioUnitario), 0)
    FROM Ventas V
    INNER JOIN DetalleVentas DV
        ON V.ID_Venta = DV.ID_Venta
    WHERE V.ID_Cliente = @ID_Cliente;

    RETURN @Total;
END;
Consulta de prueba:
SELECT dbo.TotalFacturadoCliente(1) AS TotalFacturado;
```

3. Inline Table-Valued Function, ITVF

Enunciado: Crear una ITVF que devuelva los clientes que nunca realizaron ventas y cuyo límite de crédito sea superior al promedio general de límite de crédito de todos los clientes.

La función debe devolver:

ID_Cliente
NombreCliente
LimiteCredito

Restricciones:

- Debe ser una Inline Table-Valued Function.
- No puede usar BEGIN...END.
- No puede usar tabla de retorno declarada manualmente.
- No puede usar LEFT JOIN.
- Debe usar NOT EXISTS.
- Debe usar una subconsulta para calcular el promedio.

Resultado:

```
CREATE FUNCTION dbo.ClientesSinVentasCreditoAlto()
RETURNS TABLE
AS
```

```

RETURN
(
    SELECT
        C.ID_Cliente,
        C.NombreCliente,
        C.LimiteCredito
    FROM Clientes C
    WHERE NOT EXISTS (
        SELECT 1
        FROM Ventas V
        WHERE V.ID_Cliente = C.ID_Cliente
    )
    AND C.LimiteCredito > (
        SELECT AVG(LimiteCredito)
        FROM Clientes
    )
);
Consulta de prueba:
SELECT *
FROM dbo.ClientesSinVentasCreditoAlto();
    
```

4. Multi-Statement Table-Valued Function, MSTVF

Enunciado: Crear una MSTVF que reciba dos parámetros de stock y devuelva los productos clasificados según su nivel de inventario.

La función debe recibir:

@StockCritico
@StockMedio

Debe devolver:

ID_Producto
NombreProducto
Stock
EstadoStock

Reglas:

Si Stock = 0, EstadoStock = 'Sin stock'
 Si Stock > 0 y Stock <= @StockCritico, EstadoStock = 'Crítico'
 Si Stock > @StockCritico y Stock <= @StockMedio, EstadoStock = 'Medio'
 Si Stock > @StockMedio, EstadoStock = 'Normal'

Restricciones:

- Debe ser una MSTVF.
- Debe usar tabla de retorno.
- Debe usar lógica condicional.

- Si @StockCritico es menor que 0 o @StockMedio es menor que @StockCritico, debe devolver la tabla vacía.
- No se pueden modificar datos de las tablas base.
- No se puede usar cursor.

Resultado:

```
CREATE FUNCTION dbo.ProductosPorEstadoStock (
    @StockCritico INT,
    @StockMedio INT
)
RETURNS @Resultado TABLE (
    ID_Producto INT,
    NombreProducto VARCHAR(100),
    Stock INT,
    EstadoStock VARCHAR(20)
)
AS
BEGIN
    IF @StockCritico < 0 OR @StockMedio < @StockCritico
    BEGIN
        RETURN;
    END;

    INSERT INTO @Resultado (
        ID_Producto,
        NombreProducto,
        Stock,
        EstadoStock
    )
    SELECT
        ID_Producto,
        NombreProducto,
        Stock,
        CASE
            WHEN Stock = 0 THEN 'Sin stock'
            WHEN Stock > 0 AND Stock <= @StockCritico THEN 'Crítico'
            WHEN Stock > @StockCritico AND Stock <= @StockMedio THEN 'Medio'
            ELSE 'Normal'
        END AS EstadoStock
    FROM Productos;

    RETURN;
END;
Consulta de prueba:
SELECT *
FROM dbo.ProductosPorEstadoStock(5, 20);
```

5. Multi-Statement Table-Valued Function, MSTVF

Enunciado: Crear una MSTVF que reciba un rango de fechas y devuelva un resumen de facturación por cliente, incluyendo una categoría según el monto facturado.

La función debe recibir:

@FechaDesde
@FechaHasta

Debe devolver:

ID_Cliente
NombreCliente
MontoFacturado
CategoriaCliente

Reglas:

Si el cliente no facturó en el período, CategoriaCliente = 'Sin ventas'
Si facturó hasta 100000, CategoriaCliente = 'Bajo'
Si facturó más de 100000 y hasta 500000, CategoriaCliente = 'Medio'
Si facturó más de 500000, CategoriaCliente = 'Alto'

Restricciones:

- Debe ser una MSTVF.
- Debe usar tabla de retorno.
- Debe incluir clientes aunque no tengan ventas en el período.
- Debe usar al menos un INSERT sobre la tabla de retorno.
- Puede usar UPDATE sobre la tabla de retorno.
- No puede modificar tablas base.
- No puede usar TOP.
- Si @FechaDesde es mayor que @FechaHasta, debe devolver la tabla vacía.

Resultado:

```
CREATE FUNCTION dbo.ResumenFacturacionClientes (  
    @FechaDesde DATE,  
    @FechaHasta DATE  
)  
RETURNS @Resultado TABLE (  
    ID_Cliente INT,  
    NombreCliente VARCHAR(100),  
    MontoFacturado DECIMAL(12,2),  
    CategoriaCliente VARCHAR(20)  
)  
AS  
BEGIN  
    IF @FechaDesde > @FechaHasta  
        BEGIN  
            RETURN;  
        END;  
  
    INSERT INTO @Resultado (  
        ID_Cliente,  
        NombreCliente,  
        MontoFacturado,  
        CategoriaCliente
```

```

)
SELECT
    C.ID_Cliente,
    C.NombreCliente,
    COALESCE(SUM(DV.Cantidad * DV.PrecioUnitario), 0) AS MontoFacturado,
    'Pendiente' AS CategoriaCliente
FROM Clientes C
LEFT JOIN Ventas V
    ON C.ID_Cliente = V.ID_Cliente
    AND V.Fecha >= @FechaDesde
    AND V.Fecha <= @FechaHasta
LEFT JOIN DetalleVentas DV
    ON V.ID_Venta = DV.ID_Venta
GROUP BY
    C.ID_Cliente,
    C.NombreCliente;

UPDATE @Resultado
SET CategoriaCliente =
    CASE
        WHEN MontoFacturado = 0 THEN 'Sin ventas'
        WHEN MontoFacturado <= 100000 THEN 'Bajo'
        WHEN MontoFacturado <= 500000 THEN 'Medio'
        ELSE 'Alto'
    END;

RETURN;
END;

```

Consulta de prueba:

```

SELECT *
FROM dbo.ResumenFacturacionClientes('2026-01-01', '2026-12-31');

```

Stored Procedure: registrar una venta con validación de stock

Enunciado: Crear un procedimiento almacenado que registre una venta de un producto a un cliente.

Tablas:

Clientes(ID_Cliente, NombreCliente)

Productos(ID_Producto, NombreProducto, Stock, Precio)

Ventas(ID_Venta, ID_Cliente, Fecha)

DetalleVentas(ID_Venta, ID_Producto, Cantidad, PrecioUnitario)

El procedimiento debe recibir:

@ID_Cliente

@ID_Producto

@Cantidad

Debe realizar:

1. Validar que el cliente exista.
2. Validar que el producto exista.
3. Validar que la cantidad sea mayor que cero.
4. Validar que haya stock suficiente.
5. Insertar la venta en la tabla Ventas.
6. Insertar el detalle en DetalleVentas.
7. Descontar el stock del producto.

Restricciones:

- Debe usar BEGIN TRAN, COMMIT y ROLLBACK.
- Debe usar TRY...CATCH.
- Debe usar SCOPE_IDENTITY() para recuperar el ID de la venta insertada.
- Si falla una validación, debe deshacer la transacción y mostrar un error.
- No se puede permitir que el stock quede negativo.

Resultado:

```
CREATE PROCEDURE dbo.RegistrarVenta
    @ID_Cliente INT,
    @ID_Producto INT,
    @Cantidad INT
AS
BEGIN
    BEGIN TRY
        BEGIN TRAN;

        IF NOT EXISTS (
            SELECT 1
            FROM Clientes
            WHERE ID_Cliente = @ID_Cliente
        )
```



```

BEGIN
    ROLLBACK TRAN;
    THROW 50001, 'El cliente no existe.', 1;
END;

IF NOT EXISTS (
    SELECT 1
    FROM Productos
    WHERE ID_Producto = @ID_Producto
)
BEGIN
    ROLLBACK TRAN;
    THROW 50002, 'El producto no existe.', 1;
END;

IF @Cantidad <= 0
BEGIN
    ROLLBACK TRAN;
    THROW 50003, 'La cantidad debe ser mayor que cero.', 1;
END;

IF EXISTS (
    SELECT 1
    FROM Productos
    WHERE ID_Producto = @ID_Producto
    AND Stock < @Cantidad
)
BEGIN
    ROLLBACK TRAN;
    THROW 50004, 'Stock insuficiente.', 1;
END;

DECLARE @PrecioUnitario DECIMAL(10,2);
DECLARE @ID_Venta INT;

SELECT @PrecioUnitario = Precio
FROM Productos
WHERE ID_Producto = @ID_Producto;

INSERT INTO Ventas (ID_Cliente, Fecha)
VALUES (@ID_Cliente, GETDATE());

SET @ID_Venta = SCOPE_IDENTITY();

INSERT INTO DetalleVentas (
    ID_Venta,
    ID_Producto,
    Cantidad,
    PrecioUnitario
)
VALUES (
    @ID_Venta,
    @ID_Producto,
    @Cantidad,
    @PrecioUnitario
);

UPDATE Productos
SET Stock = Stock - @Cantidad
WHERE ID_Producto = @ID_Producto;
    
```

```

        COMMIT TRAN;
    END TRY
    BEGIN CATCH
        IF @@TRANCOUNT > 0
            ROLLBACK TRAN;

        THROW;
    END CATCH
END;
Ejemplo de ejecución:
EXEC dbo.RegistrarVenta
    @ID_Cliente = 1,
    @ID_Producto = 3,
    @Cantidad = 2;

```

Stored Procedure: transferir saldo entre cuentas

Enunciado: Crear un procedimiento almacenado que transfiera saldo desde una cuenta origen hacia una cuenta destino.

Tabla: Cuentas(ID_Cuenta, Titular, Saldo)

El procedimiento debe recibir:

```

@CuentaOrigen
@CuentaDestino
@Monto

```

Debe realizar:

1. Validar que la cuenta origen exista.
2. Validar que la cuenta destino exista.
3. Validar que ambas cuentas sean distintas.
4. Validar que el monto sea mayor que cero.
5. Validar que la cuenta origen tenga saldo suficiente.
6. Restar el monto de la cuenta origen.
7. Sumar el monto a la cuenta destino.

Restricciones:

- Debe usar BEGIN TRAN, COMMIT y ROLLBACK.
- Debe usar TRY...CATCH.
- Si alguna validación falla, debe deshacer la transacción y mostrar un error.
- No se puede permitir que una cuenta quede con saldo negativo.
- No se puede usar cursor.

Resultado:

```

CREATE PROCEDURE dbo.TransferirSaldo
    @CuentaOrigen INT,
    @CuentaDestino INT,
    @Monto DECIMAL(12,2)
AS
BEGIN
    BEGIN TRY
        BEGIN TRAN;

        IF NOT EXISTS (
            SELECT 1
            FROM Cuentas
            WHERE ID_Cuenta = @CuentaOrigen
        )
        BEGIN
            ROLLBACK TRAN;
            THROW 50001, 'La cuenta origen no existe.', 1;
        END;

        IF NOT EXISTS (
            SELECT 1
            FROM Cuentas
            WHERE ID_Cuenta = @CuentaDestino
        )
        BEGIN
            ROLLBACK TRAN;
            THROW 50002, 'La cuenta destino no existe.', 1;
        END;

        IF @CuentaOrigen = @CuentaDestino
        BEGIN
            ROLLBACK TRAN;
            THROW 50003, 'La cuenta origen y destino no pueden ser la misma.', 1;
        END;

        IF @Monto <= 0
        BEGIN
            ROLLBACK TRAN;
            THROW 50004, 'El monto debe ser mayor que cero.', 1;
        END;

        IF EXISTS (
            SELECT 1
            FROM Cuentas
            WHERE ID_Cuenta = @CuentaOrigen
            AND Saldo < @Monto
        )
        BEGIN
            ROLLBACK TRAN;
            THROW 50005, 'Saldo insuficiente.', 1;
        END;

        UPDATE Cuentas
        SET Saldo = Saldo - @Monto
        WHERE ID_Cuenta = @CuentaOrigen;

        UPDATE Cuentas
        SET Saldo = Saldo + @Monto
        WHERE ID_Cuenta = @CuentaDestino;
    
```

```
        COMMIT TRAN;  
    END TRY  
    BEGIN CATCH  
        IF @@TRANCOUNT > 0  
            ROLLBACK TRAN;  
  
        THROW;  
    END CATCH  
END;
```

Ejemplo de ejecución:

```
EXEC dbo.TransferirSaldo  
    @CuentaOrigen = 1,  
    @CuentaDestino = 2,  
    @Monto = 15000;
```

Ejercicio de práctica: Proveedores con compras superiores al promedio

Enunciado para desarrollar en SQL Code: Crear un esquema llamado Compras.

Dentro del esquema Compras, crear dos tablas:

Proveedores: que almacene información de los proveedores.

OrdenesCompra: que almacene las órdenes de compra realizadas a cada proveedor.

Las tablas deben tener la siguiente estructura mínima:

Proveedores: ID_Proveedor, NombreProveedor, Rubro

OrdenesCompra: ID_Orden, ID_Proveedor, Fecha, Importe

Insertar:

- 4 registros en la tabla Proveedores.
- 8 registros en la tabla OrdenesCompra.
- Algunos proveedores deben tener más de una orden de compra.

Consulta solicitada:

Escribir una consulta que devuelva los proveedores cuyo monto total comprado sea superior al promedio de compras acumuladas por proveedor.

La consulta debe devolver:

ID_Proveedor
NombreProveedor
TotalComprado

Restricciones:

- Debe usarse GROUP BY.
- Debe usarse una subconsulta.
- No se puede usar TOP.
- No se puede usar MAX ni MIN.
- Deben considerarse únicamente proveedores con órdenes de compra registradas.

```
CREATE SCHEMA Compras;  
GO
```

```
CREATE TABLE Compras.Proveedores (  
    ID_Proveedor INT PRIMARY KEY,  
    NombreProveedor VARCHAR(100),  
    Rubro VARCHAR(100)  
);
```

```
CREATE TABLE Compras.OrdenesCompra (
    ID_Orden INT PRIMARY KEY,
    ID_Proveedor INT,
    Fecha DATE,
    Importe DECIMAL(12,2),
    FOREIGN KEY (ID_Proveedor)
        REFERENCES Compras.Proveedores(ID_Proveedor)
);
```

```
INSERT INTO Compras.Proveedores VALUES
(1, 'Proveedor A', 'Tecnología'),
(2, 'Proveedor B', 'Librería'),
(3, 'Proveedor C', 'Mobiliario'),
(4, 'Proveedor D', 'Servicios');
```

```
INSERT INTO Compras.OrdenesCompra VALUES
(1, 1, '2026-01-10', 150000),
(2, 1, '2026-02-15', 200000),
(3, 2, '2026-01-20', 50000),
(4, 2, '2026-03-05', 70000),
(5, 3, '2026-02-01', 300000),
(6, 3, '2026-04-12', 250000),
(7, 4, '2026-03-18', 90000),

(8, 4, '2026-05-22', 110000);
```

Consulta:

```
SELECT
    P.ID_Proveedor,
    P.NombreProveedor,
    SUM(OC.Importe) AS TotalComprado
FROM Compras.Proveedores P
INNER JOIN Compras.OrdenesCompra OC
    ON P.ID_Proveedor = OC.ID_Proveedor
GROUP BY
    P.ID_Proveedor,
    P.NombreProveedor
HAVING
    SUM(OC.Importe) >
    (
        SELECT AVG(TotalPorProveedor)
        FROM
            (
                SELECT
                    SUM(OC2.Importe) AS TotalPorProveedor
                FROM Compras.OrdenesCompra OC2
                GROUP BY OC2.ID_Proveedor
            ) AS Totales
    );
```

Resultado esperado con esos datos:

ID_Proveedor	NombreProveedor	TotalComprado
3	Proveedor C	550000.00

Porque los totales por proveedor son:

Proveedor A: 350000

Proveedor B: 120000

Proveedor C: 550000

Proveedor D: 200000

El promedio de compras acumuladas por proveedor es:

$$(350000 + 120000 + 550000 + 200000) / 4 = 305000$$

Vista de resumen de ventas por cliente

Enunciado:

A partir de las siguientes tablas:

Clientes(ID_Cliente, NombreCliente, Provincia)

Ventas(ID_Venta, ID_Cliente, Fecha)

DetalleVentas(ID_Venta, ID_Producto, Cantidad, PrecioUnitario)

Crear una vista llamada vw_ResumenVentasClientes que devuelva el monto total vendido por cliente.

La vista debe devolver:

ID_Cliente

NombreCliente

Provincia

CantidadVentas

MontoTotalVendido

El monto total vendido debe calcularse como:

$\text{SUM}(\text{DetalleVentas.Cantidad} * \text{DetalleVentas.PrecioUnitario})$

Restricciones:

- Debe usarse CREATE VIEW.
- La vista no debe recibir parámetros.
- Debe incluir solo clientes que tengan ventas registradas.
- Debe usarse GROUP BY.
- No se puede usar ORDER BY dentro de la vista.
- No se puede modificar información dentro de la vista.

Solución:

```
CREATE VIEW dbo.vw_ResumenVentasClientes
AS
SELECT
    C.ID_Cliente,
    C.NombreCliente,
    C.Provincia,
    COUNT(DISTINCT V.ID_Venta) AS CantidadVentas,
    SUM(DV.Cantidad * DV.PrecioUnitario) AS MontoTotalVendido
FROM Clientes C
INNER JOIN Ventas V
    ON C.ID_Cliente = V.ID_Cliente
INNER JOIN DetalleVentas DV
    ON V.ID_Venta = DV.ID_Venta
GROUP BY
    C.ID_Cliente,
    C.NombreCliente,
    C.Provincia;
```

Consulta de prueba:

```
SELECT *
FROM dbo.vw_ResumenVentasClientes;
```


Trigger para evitar stock negativo

Enunciado:

A partir de la siguiente tabla: Productos(ID_Producto, NombreProducto, Stock), crear un trigger que impida que el stock de un producto quede en negativo luego de una operación de actualización.

El trigger debe ejecutarse sobre la tabla Productos cuando se realice un UPDATE.

Restricciones:

- Debe ser un trigger AFTER UPDATE.
- Debe validar los registros afectados por la actualización.
- Si algún producto queda con Stock < 0, debe cancelar la operación.
- Debe mostrar un error.
- Debe usar la tabla lógica inserted.
- No debe usar cursor.
- Debe funcionar aunque el UPDATE afecte a más de un producto.

Solución:

```
CREATE TRIGGER dbo.trg_Productos_ValidarStockNegativo
ON dbo.Productos
AFTER UPDATE
AS
BEGIN
    SET NOCOUNT ON;

    IF EXISTS (
        SELECT 1
        FROM inserted
        WHERE Stock < 0
    )
    BEGIN
        RAISERROR('El stock no puede quedar en negativo.', 16, 1);
        ROLLBACK TRANSACTION;
        RETURN;
    END;
END;

Consulta de prueba válida:
UPDATE Productos
SET Stock = Stock - 2
WHERE ID_Producto = 1;
Consulta de prueba inválida:
UPDATE Productos
SET Stock = -5
WHERE ID_Producto = 1;
```